Mini-PROJET

Système de Gestion d'une Boutique en Ligne

ShopFlow

Durée du projet	3 semaines
Travail	Binôme ou individuel
Rendu	Archive ZIP + Rapport PDF + Exposé
Backend (sur 15)	Spring Boot 3 — Jakarta EE · MVC · JPA · Lombok
Frontend (sur 5)	Next.js ou Angular
Note totale	20 points

Année universitaire 2025 – 2026

1. Contexte et Objectifs

Dans ce projet, vous devez concevoir et développer ShopFlow, une plateforme e-commerce complète permettant à des vendeurs de publier leurs produits et à des clients d'effectuer des achats en ligne. Ce sujet couvre l'ensemble de la chaîne de valeur d'une boutique numérique moderne : catalogue produits, panier, commandes, paiement simulé et gestion des stocks.

1.1 Présentation du système

ShopFlow est une marketplace B2C (Business to Consumer) comportant trois profils utilisateurs :

- ADMIN : gestion globale de la plateforme, des utilisateurs et des catégories
- SELLER (Vendeur) : gestion de sa boutique, de ses produits et de ses commandes reçues
- CUSTOMER (Client) : navigation dans le catalogue, gestion du panier, passage de commandes

1.2 Objectifs pédagogiques

- Concevoir une API REST complète avec Spring Boot 3 et Jakarta EE
- Modéliser des relations JPA complexes (ManyToMany, héritage, composition)
- Implémenter des règles métier réelles : stock, statut de commande, remises
- Gérer l'authentification et les autorisations par rôle (Spring Security + JWT)
- Construire une interface utilisateur dynamique consommant l'API
- Produire une documentation Swagger/OpenAPI exploitable

2. Spécifications Fonctionnelles

2.1 Gestion du Catalogue Produits

- Ajouter, modifier, désactiver un produit (SELLER/ADMIN)
- Associer un produit à une ou plusieurs catégories
- Gérer les variantes d'un produit : taille, couleur, avec stock individuel par variante
- Uploader des images (URL distante ou champ texte simulant l'upload)
- Recherche plein texte : nom, description, catégorie, prix min/max
- Trier par prix, popularité, nouveautés ; pagination obligatoire
- Afficher les produits en promotion (prix barré + pourcentage de remise)

2.2 Gestion des Utilisateurs

- Inscription client avec validation (email unique, mot de passe fort)
- Authentification par JWT (access token + refresh token)
- Gestion du profil : adresses de livraison multiples, historique de commandes
- Inscription vendeur avec informations boutique (nom, logo, description)
- Activation / désactivation de compte (ADMIN)
- Réinitialisation de mot de passe par token

2.3 Panier d'Achat

- Ajouter / retirer des produits (ou variantes) dans le panier
- Modifier les quantités avec vérification du stock disponible en temps réel
- Appliquer un code promo (remise fixe ou pourcentage)
- Calculer le sous-total, frais de livraison et total TTC
- Sauvegarder le panier en base (persistant après déconnexion)

2.4 Gestion des Commandes

- Passer une commande depuis le panier (vérification stock finale)
- Choisir une adresse de livraison parmi les adresses sauvegardées
- Simuler le paiement : PENDING -> PAID -> PROCESSING -> SHIPPED -> DELIVERED
- Annuler une commande (seulement si statut PENDING ou PAID)
- Remboursement simulé en cas d'annulation
- Génération d'un numéro de commande unique (ex : ORD-2024-XXXXX)
- Notification de changement de statut (réponse API ou flag isNew)

2.5 Avis & Notation

- Laisser un avis (note 1-5 + commentaire) uniquement sur un produit acheté
- Calcul de la note moyenne automatique
- Modération des avis (ADMIN)

2.6 Tableau de Bord

- ADMIN : chiffre d'affaires global, top produits, top vendeurs, commandes récentes
- SELLER : revenus, commandes en attente, alertes de stock faible
- CUSTOMER : mes commandes en cours, mes derniers avis

3. Spécifications Techniques — Backend (15 pts)

3.1 Stack technologique imposée

Technologie	Version / Détail
Java	17 ou 21 (LTS)
Spring Boot	3.x
Jakarta EE	Jakarta Persistence 3, Jakarta Validation 3, Jakarta Bean Validation
Spring MVC	Controllers REST — @RestController, @RequestMapping
Spring Data JPA	Repositories, JPQL, Specifications, Pageable
Lombok	@Data, @Builder, @Slf4j, @RequiredArgsConstructor, @Value
Base de données	PostgreSQL (production) ou H2 en mémoire (dev/test)
Sécurité	Spring Security 6 + JWT (jjwt) — accès token + refresh token

Build	Maven (pom.xml fourni)
Documentation API	Springdoc OpenAPI 3 (Swagger UI accessible sur /swagger-ui)
Mapping DTO	MapStruct (recommandé) ou mapping manuel

3.2 Architecture en couches — obligatoire

- Controller : expose les endpoints REST, valide les entrées (@Valid), retourne les DTOs et codes HTTP
- Service : contient la logique métier, les règles de gestion, les transactions (@Transactional)
- Repository : interfaces Spring Data JPA, requêtes JPQL ou dérivées, Specifications pour les filtres complexes
- Entity : entités JPA annotées (Jakarta Persistence), relations, contraintes, lifecycle hooks
- DTO : Request / Response séparés des entités — jamais d'entité exposée directement
- Exception : classes d'exception métier custom + @ControllerAdvice globalisant la gestion des erreurs
- Mapper : conversion Entity <-> DTO (MapStruct ou manuel)

3.3 Modèle de données (entités minimales)

- User : id, email, motDePasse, prenom, nom, role (ADMIN/SELLER/CUSTOMER), actif, dateCreation
- SellerProfile : id, user (1-1), nomBoutique, description, logo, note
- Address : id, user, rue, ville, codePostal, pays, principal (boolean)
- Category : id, nom, description, parent (auto-référence pour sous-catégories)
- Product : id, seller, nom, description, prix, prixPromo, stock, actif, dateCreation, categories (M-M), images
- ProductVariant : id, product, attribut (ex: Taille), valeur (ex: M), stockSupplémentaire, prixDelta
- Cart : id, customer (1-1), lignes (1-M), dateModification
- CartItem : id, cart, product, variant (nullable), quantite
- Order : id, customer, statut (enum), numeroCommande, adresseLivraison, sousTotal, fraisLivraison, totalTTC, dateCommande, lignes
- OrderItem : id, order, product, variant, quantite, prixUnitaire
- Coupon : id, code, type (PERCENT/FIXED), valeur, dateExpiration, usagesMax, usagesActuels, actif
- Review : id, customer, product, note (1-5), commentaire, dateCreation, approuve

Les relations doivent être correctement configurées : cascade, orphanRemoval, FetchType (LAZY par défaut, EAGER avec justification).

3.4 Endpoints REST attendus

Authentification — /api/auth

- POST /api/auth/register — inscription client ou vendeur
- POST /api/auth/login — retourne access_token + refresh_token
- POST /api/auth/refresh — renouvellement du token
- POST /api/auth/logout — invalidation du refresh token

Produits — /api/products

- GET /api/products — liste paginée + filtres (categorie, prix, vendeur, promo)
- GET /api/products/{id} — détail + variantes + avis + note moyenne
- POST /api/products — créer (SELLER/ADMIN)
- PUT /api/products/{id} — modifier
- DELETE /api/products/{id} — désactiver (soft delete)
- GET /api/products/search?q= — recherche full-text
- GET /api/products/top-selling — top 10 meilleures ventes

Catégories — /api/categories

- GET /api/categories — arbre de catégories
- POST/PUT/DELETE /api/categories — gestion (ADMIN)

Panier — /api/cart

- GET /api/cart — panier du client connecté
- POST /api/cart/items — ajouter un article
- PUT /api/cart/items/{itemId} — modifier quantité
- DELETE /api/cart/items/{itemId} — retirer un article
- POST /api/cart/coupon — appliquer un code promo
- DELETE /api/cart/coupon — retirer le code promo

Commandes — /api/orders

- POST /api/orders — passer une commande depuis le panier
- GET /api/orders/{id} — détail
- GET /api/orders/my — commandes du client connecté
- PUT /api/orders/{id}/status — mettre à jour le statut (SELLER/ADMIN)
- PUT /api/orders/{id}/cancel — annuler (CUSTOMER si éligible)
- GET /api/orders — toutes les commandes (ADMIN)

Avis — /api/reviews

- POST /api/reviews — poster un avis (CUSTOMER, achat vérifié)
- GET /api/reviews/product/{productId} — avis d'un produit
- PUT /api/reviews/{id}/approve — approuver (ADMIN)

Dashboard — /api/dashboard

- GET /api/dashboard/admin — stats globales
- GET /api/dashboard/seller — stats du vendeur connecté

Coupons — /api/coupons

- POST/PUT/DELETE /api/coupons — gestion (ADMIN)
- GET /api/coupons/validate/{code} — vérifier la validité

4. Spécifications Techniques — Frontend (5 pts)

4.1 Framework au choix : Next.js ou Angular

Choisissez un seul framework et justifiez votre choix dans le rapport (ecosystème, SSR, routing, gestion d'état...).

4.2 Pages & fonctionnalités minimales

- Page d'accueil : produits en vedette, catégories, bannière promo
- Catalogue produits : grille avec filtres dynamiques (catégorie, prix, note) et recherche
- Fiche produit : images, variantes sélectionnables, avis, bouton Ajouter au panier
- Panier : liste des articles, application coupon, total, bouton Commander
- Tunnel de commande : sélection adresse, récapitulatif, confirmation
- Espace client : mes commandes + suivi statut, mes avis
- Espace vendeur : dashboard rapide, gestion produits, commandes reçues
- Authentification : login / register avec gestion JWT (stockage sécurisé)

4.3 Critères de qualité

- Design responsive (mobile-first recommandé)
- Gestion des états : chargement (skeleton/spinner), erreurs, listes vides
- Intercepteur HTTP pour injecter le Bearer token dans chaque requête
- Refresh automatique du token expiré
- Guards/middleware de route pour les pages protégées
- Composants réutilisables : ProductCard, OrderStatusBadge, RatingStars, etc.
- Optionnel (+bonus) : mode sombre, animations de transition, PWA

5. Grille d'Évaluation Détaillée

5.1 Backend — 15 points

Critère d'évaluation	Pts	Ce que nous évaluerons
Modèle de données & entités JPA	**3**	*Relations, contraintes DB, enums, lifecycle, héritage*
Repository & requêtes de données	**2**	*Filtres, pagination, JPQL, Specifications, performances*
Couche Service & logique métier	**3**	*Règles commande, stock, coupon, statut, transactions*
Couche Controller REST	**2**	*Codes HTTP corrects, validation @Valid, gestion des cas limites*
Sécurité JWT & autorisation par rôle	**3**	*Access/refresh token, filtres, @PreAuthorize par endpoint*
Qualité & documentation	**2**	*Lombok, proprete code, Swagger UI fonctionnel, README*
BONUS : Tests JUnit/Mockito	**+2**	*Tests sur la couche Service (couverture >= 70%)*
TOTAL BACKEND	**15**	+ 2 bonus

5.2 Frontend — 5 points

Critère d'évaluation	Pts	Ce que nous évaluerons
Intégration API & gestion JWT	**2**	*Appels HTTP, intercepteur, refresh token, routes gardées*
Fonctionnalités UI : catalogue, panier, commandes	**2**	*Flux complet d'achat fonctionnel*
Qualité du code & UX/UI	**1**	*Responsive, composants réutilisables, gestion d'états*
TOTAL FRONTEND	**5**	/5 points

6. Livrables & Modalités de Rendu

6.1 Contenu de l'archive ZIP

- backend/ — code source Spring Boot (sans target/)
- frontend/ — code source Next.js ou Angular (sans node_modules/ ni .next/)
- rapport.pdf — 6 à 8 pages : architecture, choix techniques, difficultés
- README.md — instructions complètes d'installation et de lancement
- shopflow.postman_collection.json — collection Postman couvrant tous les endpoints
- data.sql (optionnel) — données de démonstration pour la présentation

6.2 Contenu du rapport technique

1. Schéma d'architecture technique (couches + flux de données)
2. Description des choix d'implémentation les plus complexes
3. Difficultés rencontrées et solutions apportées
4. Répartition des tâches si binôme

6.3 Déroulement de la soutenance (10 min)

- Démonstration live du flux complet : inscription → achat → suivi commande (3 min)
- Présentation du code : entités JPA, service le plus complexe, sécurité JWT (3 min)
- Questions (4 min)

7. Conseils Méthodologiques & Ressources

Ordre de développement recommandé

5. Modéliser les entités et valider le schéma DB (H2 Console)
6. Implémenter les repositories avec des tests de requêtes simples
7. Développer les services module par module : Auth → Catalogue → Panier → Commandes
8. Ajouter la sécurité Spring Security + JWT en avant-dernière étape
9. Documenter avec Swagger au fur et à mesure

10. Développer le frontend une fois l'API backend stabilisée
11. Soigner la collection Postman avant le rendu

Ressources

- Spring Boot 3 docs : https://docs.spring.io/spring-boot/docs/current/reference/html/
- Spring Data JPA : https://docs.spring.io/spring-data/jpa/docs/current/reference/html/
- Spring Security 6 + JWT : https://docs.spring.io/spring-security/reference/
- Lombok : https://projectlombok.org/features/
- Springdoc OpenAPI : https://springdoc.org/
- MapStruct : https://mapstruct.org/documentation/stable/reference/html/
- Next.js : https://nextjs.org/docs
- Angular : https://angular.dev/overview